

KLERION

Project Handover and Technical Operations Manual

Extremely Detailed Engineering, Product, Operations and Commercial Handover

Repository	Lugbinlaye/Klerion (private GitHub repository)
Default branch	main
Repository state reviewed	Commit 40c995c4ace44714c41c9f03b8c614d987446d54
Handover date	6 August 2026
Document version	1.0
Prepared for	Incoming engineering, product, operations, security and commercial teams

Confidentiality: This document describes a private software repository, its architecture, security assumptions, operational procedures and unresolved risks. Distribute only to authorised stakeholders.

Klerion - the operating system for organised companies.

0. Document Control and Handover Basis

This manual is the authoritative handover snapshot for the Klerion project at the date and repository commit stated on the cover. It is designed to let a new technical or product team understand what exists, what is live, what remains demonstrational, how to run the software, how to protect tenant data, and how to continue delivery without reopening resolved branch conflicts.

Field	Value
Document owner	Klerion project owner / repository administrator
Primary technical audience	Engineering lead, backend engineers, frontend engineers, QA, DevOps/SRE, security reviewer
Secondary audience	Product manager, operations lead, sales engineering, implementation partner, executive sponsor
Source of truth	Code on main, current CI workflows, merged PR history, database migrations, domain and route implementations
Known documentation caveat	DEVELOPMENT.md and PROGRESS.md predate the Workforce Lifecycle and Customer Intelligence merges; this manual reconciles them with current code
Update expectation	Revise at each production release, major schema migration, security-model change, or material team handover

0.1 How to Use This Manual

- **New engineer:** Read Sections 1, 4, 5, 10, 11 and 17 before changing code.
- **Engineering lead:** Read Sections 3, 6, 12, 14, 18 and 22 before planning the next sprint.
- **DevOps/SRE:** Read Sections 9, 15, 16, 19 and 20 before deploying or handling incidents.
- **Security reviewer:** Read Sections 7, 8, 14, 19 and 21 before approving a pilot.
- **Product/commercial team:** Read Sections 1, 2, 3, 13 and 23 to understand the sellable scope and the difference between live and preview functionality.
- **Incoming project owner:** Use Sections 24, 25 and 26 as the formal handover and acceptance checklist.

0.2 Source Files Reviewed

Area	Primary source paths
Project operation	DEVELOPMENT.md; PROGRESS.md; package.json
Team plan	docs/TEAM_EXECUTION_PLAN_6_DEVELOPERS.md
API composition	apps/api/src/server.ts; apps/api/src/context.ts; apps/api/package.json
Authorization	modules/platform/identity/src/permission.ts
Workforce Lifecycle	apps/api/src/domains/workforce-lifecycle/**; apps/api/src/routes/workforce-lifecycle.ts;

Area	Primary source paths
	packages/persistence/migrations/0002_workforce_lifecycle.sql
Customer Intelligence	apps/api/src/domains/customer-intelligence/**; apps/api/src/routes/customer-intelligence.ts; packages/persistence/migrations/0003_customer_intelligence.sql
Web console	apps/web/src/App.tsx; apps/web/src/components/Shell.tsx; apps/web/package.json
CI	.github/workflows/ci.yml; .github/workflows/web-ci.yml
Branch reconciliation	Merged PR #17 and merge commit ad86feaac68ca64946ada905f57a45f6ac5ccfbe

Contents

1. 1. Executive Handover Summary
 2. 2. Product Vision and Commercial Positioning
 3. 3. Current Product Truth and Feature Status
 4. 4. System Architecture Overview
 5. 5. Repository Structure and Ownership
 6. 6. Technology Stack and Engineering Conventions
 7. 7. Tenancy, Identity and Authorization
 8. 8. Audit, Integrity and Accountability
 9. 9. Runtime Topology and End-to-End Flows
 10. 10. Domain Module Handover
 11. 11. Complete API Reference
 12. 12. Database, Persistence and Migrations
 13. 13. Web Console Handover
 14. 14. Security and Privacy Posture
 15. 15. Local Development Setup
 16. 16. Configuration and Environment Management
 17. 17. Testing, CI and Quality Gates
 18. 18. Git Branch, PR and Merge History
 19. 19. Deployment and Release Runbook
 20. 20. Operations, Monitoring and Incident Response
 21. 21. Known Limitations, Risks and Technical Debt
 22. 22. Six-Developer Ownership Model
 23. 23. Commercial Roadmap and Recommended Delivery Order
 24. 24. Troubleshooting Guide
 25. 25. Formal Handover Checklist
 26. 26. Acceptance Sign-Off
- Appendices A-H

1. Executive Handover Summary

Klerion is a multi-tenant administrative operations platform intended to unify customer flow, workforce administration, internal workflows, case resolution and executive visibility. The current repository is a working modular-monolith foundation with a React company console, Fastify API, strict TypeScript packages, PostgreSQL-compatible persistence, role-based access control and a tamper-evident audit trail.

At handover, two substantial commercial vertical slices are active in addition to the original foundation: Workforce Lifecycle (leave plus onboarding/offboarding) and Customer Resolution and Intelligence (cases, complaints, SLA tracking and executive summaries). These features were reconciled into main with green backend and web CI. Several older branches contain valuable but incompatible implementations; their complete histories are preserved in main ancestry, but their code was intentionally not activated because direct merges would have deleted or replaced working platform modules.

Critical truth: Do not assume every screen visible in the console is backed by a production API. Authentication, users/roles, audit, appointments, leave/lifecycle, cases and executive summaries have live API paths. The overview dashboard, queue, recruitment and general reports still contain preview or foundation behaviour unless separately completed after this handover.

1.1 Handover State at a Glance

Dimension	Current state	Handover implication
Repository	Private; default branch main; latest reviewed head 40c995c...	All new work must branch from current main, not historic developer branches.
Backend	Fastify + TypeScript modular monolith	Maintain domain boundaries; avoid cross-domain table access.
Frontend	React 19 + Vite; separate pnpm toolchain	Both API and web server are required locally.
Persistence	In-memory fallback and PostgreSQL/PGLite adapters	Production requires DATABASE_URL; in-memory data is ephemeral.
Security	Tenant context, JWT, RBAC, server-side permission guards	No row-level security; tenant filters are application-enforced.
Audit	Per-tenant hash-chained events	Never edit audit history; append corrective events.
Live domains	Tenancy, identity, role administration, appointments, workforce lifecycle, cases/intelligence, audit	Use these as integration patterns for remaining domains.
Preview domains	Queue, recruitment, general overview/reports and some counts/badges	Do not sell as completed without explicit demo disclosure.
Branch state	Every original branch is an ancestor of main; incompatible work preserved history-only	Cherry-pick or reconstruct feature-by-feature rather than re-merging branches.
CI	Backend install/typecheck/lint/test/build; web install/typecheck/static-check/build	No PR should merge until applicable pipelines are green.

1.2 Immediate Priorities for the Incoming Team

27. **Update repository documentation.** PROGRESS.md and parts of DEVELOPMENT.md do not reflect the active Workforce Lifecycle and Customer Intelligence modules.
28. **Resolve organisation/branch/service foundations.** Branch and service catalogue remain prerequisites for a real queue, capacity routing and branch analytics.
29. **Decide package-manager convergence.** The backend uses npm workspaces while apps/web uses pnpm separately.
30. **Add web tests and real linting.** The web lint script is currently TypeScript checking rather than ESLint.
31. **Introduce observability.** Structured logs, request correlation, metrics, traces and error tracking are not production-ready.
32. **Establish migration governance.** Migrations are ordered files, but a formal schema migration history mechanism should be confirmed and tested for long-lived environments.
33. **Define pilot scope.** Select one sellable product pack and remove all misleading preview indicators from the pilot journey.

2. Product Vision and Commercial Positioning

Klerion is positioned as an operating system for organised companies: one administrative platform that connects customer visits, employee processes, approvals, service requests, complaints, audit evidence and executive information. Its commercial value is not the count of isolated features; it is the reduction of fragmented administrative work, repeated data entry, unclear ownership and invisible operational delay.

2.1 Target Markets

Market	Primary pain solved	Best entry package
Banks and financial institutions	Branch congestion, appointment routing, complaint SLA, auditability	Klerion Flow + Resolve & Insight
Hospitals and clinics	Appointments, arrival, waiting, staff availability, service communication	Klerion Flow + Workforce
Government service centres	Citizen visits, service routing, case accountability, transparency	Klerion Flow + Resolve & Insight
Schools and universities	Administrative requests, employee attendance, service appointments	Workforce + ServiceOps
Retail and hospitality groups	Multi-location staffing, complaints, maintenance and branch visibility	Workforce + Resolve & Insight
Professional services firms	Consultation scheduling, employee workflows, approvals and client cases	Flow + ServiceOps
SMEs	Replace spreadsheets and messaging-based administration	Workforce + ServiceOps
Large enterprises	Central governance with branch/department delegation	Full modular suite

2.2 Commercial Product Families

Product family	Included capability	Repository status
Klerion Flow	Branches, services, appointments, remote check-in, queues, notifications	Appointments active; branch/service/queue/notification work

Product family	Included capability	Repository status
		remains incomplete on main.
Klerion Workforce	Employee records, attendance, leave, availability, onboarding/offboarding	Leave and lifecycle active; employee records and attendance are preserved in historic branch work but not active.
Klerion ServiceOps	Forms, requests, approvals, internal service desk	Not active on main; Developer-5 history is preserved but unrelated/incompatible.
Klerion Resolve & Insight	Cases, complaint SLA, executive operational intelligence	Cases and executive summary active; broader cross-domain analytics remains limited.

3. Current Product Truth and Feature Status

The following matrix is the most important product handover artefact. It distinguishes code that is active on main from screen-level demonstrations and from work preserved only in branch history.

Capability	Status	Evidence / scope	Important limitation
Organisation creation	Live	POST /tenants plus first user signup flow	Production foundation
User signup/login	Live	JWT session generation and verification	Production foundation
Users and roles	Live	Tenant user listing and owner-only role updates	Last owner cannot be demoted
Audit trail	Live	Hash-chained per-tenant events and integrity verification	Owner/tenant management access
Appointments	Live foundation	Book, list, check-in, complete/cancel/no-show domain lifecycle	No branch/service catalogue or availability engine on main
Leave and availability	Live	Requests, balances, overlap/balance checks, approval/rejection/cancellation	Uses user IDs rather than canonical employee domain
Onboarding/offboarding	Live	Plans, default/custom steps, completion and cancellation	No direct identity deprovisioning automation
Cases and complaints	Live	Create, list, assign, status transitions, comments and SLA state	Customer access portal not implemented
Executive summary	Live limited	Calculated from active cases and appointments	Not yet a full enterprise BI/event platform
Overview dashboard	Preview/partial	UI metrics still contain representative content	Do not present as fully live
Queue and remote check-in	Preview on main	Historic Queue-management	Direct branch was destructive

Capability	Status	Evidence / scope	Important limitation
		branch preserved only	and not activated
Employee records and attendance	Not active on main	Historic Developer-3 branch preserved only	Must be selectively integrated and revalidated
Workflow and service desk	Not active on main	Historic Developer-5 history preserved	No common original history; requires reconstruction
Recruitment	Preview	UI-only foundation	No talent domain API or persistence
Reports	Foundation/preview	Placeholder/foundation view	Use executive summary for real metrics

3.1 Meaning of Status Labels

- **Live:** An authenticated route, domain/service implementation and storage path exist on main.
- **Live foundation:** A real vertical slice exists, but major commercial dependencies or workflows remain incomplete.
- **Preview/partial:** The screen can be demonstrated, but some or all records/actions are representative, local or non-persistent.
- **Historic only:** Code exists in Git ancestry but was not activated in the production tree.
- **Not started:** No reliable implementation exists on main or in an approved integration branch.

4. System Architecture Overview

Klerion follows a modular-monolith architecture. The runtime is a single Fastify API process composed from platform and domain modules, with a separate browser application. This architecture is appropriate for the current team because it preserves transaction simplicity and operational cost while still allowing explicit domain ownership.

```

Browser (React/Vite on :5173)
  | /api proxy, bearer token, tenant slug
  v
Fastify API (:3000)
- tenant context plugin
- authentication guard
- permission guard
- route modules
  |
  |--> platform: tenancy / identity / audit
  |--> domains: appointments / workforce lifecycle / customer intelligence
  |
  |--> in-memory repositories (development fallback)
  |--> PostgreSQL repositories (DATABASE_URL)
      |
      |--> ordered SQL migrations 0001, 0002, 0003

```

4.1 Layering Rules

- **Platform layer:** Tenancy, identity and audit provide reusable foundations. They must not import business domains.
- **Domain layer:** Business rules, state transitions, validation errors and repository contracts.
- **Application/service layer:** Coordinates repositories, rules and read-model calculations.
- **HTTP layer:** Validates external input, applies permissions, maps domain errors to HTTP responses and emits audit events.
- **Persistence layer:** Implements repository contracts for PostgreSQL and maps database records to domain types.

- **Web layer:** Calls authenticated APIs and presents a responsive company console. It must clearly distinguish preview from live data.

4.2 Architectural Strengths

- Strict TypeScript and project references on backend workspaces.
- Repository interfaces support in-memory and Postgres implementations.
- Fast API tests through `app.inject()` without live sockets.
- PGlite tests exercise real PostgreSQL semantics.
- Tenant-aware repositories and permission guards are established patterns.
- Audit actions are integrated at route boundaries.

4.3 Architectural Weaknesses

- Shared server/context/permission registries remain merge hotspots.
- Frontend is not integrated into the root TypeScript and ESLint workspace.
- No formal event bus or cross-domain contract package is active.
- No OpenAPI-generated contract or schema-first validation.
- No row-level security or database-enforced tenant policy.
- No observability stack, background-job framework, cache or provider abstraction for external notifications.

5. Repository Structure and Ownership

Klerion/	
apps/	
api/	Fastify API and active domain wiring
web/	React 19 + Vite company console
modules/	
platform/	
tenancy/	Tenant entity/context/repositories
identity/	Users, passwords, JWT, RBAC
audit/	Hash-chained audit log
domains/	
branch-flow/	Appointment domain
packages/	
persistence/	Drizzle/Postgres adapters and migrations
tsconfig/	Shared TypeScript base
eslint-config/	Shared backend lint configuration
apps/api/src/domains/	
workforce-lifecycle/	Leave and lifecycle vertical slice
customer-intelligence/	Case/SLA and executive read model
docs/	Planning and handover documentation

5.1 Important Shared Files

File	Responsibility	Change policy
package.json	Backend workspaces and root scripts	Integration maintainer only during parallel sprints
apps/api/src/server.ts	Registers protected and public route modules	Additive composition; never remove existing routes during feature merges
apps/api/src/context.ts	Builds in-memory/Postgres services	Preserve every active repository and service when reconciling branches
modules/platform/identity/src/permission.ts	Role-to-permission map	Security review required for additions or role changes
apps/web/src/App.tsx	Top-level route/view composition	Prefer feature registry refactor before more parallel work

File	Responsibility	Change policy
apps/web/src/components/Shell.tsx	Navigation, session display and command palette	Shared UX ownership; badges must not misrepresent live status
packages/persistence/migrations/**	Database evolution	Never rewrite applied migrations; add new ordered files
.github/workflows/**	Quality gates	DevOps/integration owner approval

6. Technology Stack and Engineering Conventions

Layer	Technology	Current use
Language	TypeScript, ESM	Strict backend project references; standalone web tsconfig
API	Fastify 5	Health, tenant, auth and protected domain routes
Frontend	React 19.2 + React DOM 19.2	Single-page company console
Build	TypeScript compiler + Vite 8	Backend emits package dist; web bundles separately
Persistence	PostgreSQL + Drizzle ORM	Production adapter and PGlite test environment
Auth	scrypt + jose JWT HS256	Password hashing and signed sessions
Testing	Node built-in test runner	Workspace tests and Fastify injection tests
Icons	lucide-react	Console navigation and actions
Package managers	npm + pnpm	Backend root workspaces and separate apps/web install
CI	GitHub Actions, Node 24	Backend and path-filtered web workflows

6.1 Coding Expectations

- Use ESM imports with .js suffixes in TypeScript source where required by NodeNext output.
- Model business errors as explicit domain error types, then map them at HTTP boundaries.
- Keep every tenant-owned repository method scoped by tenantId.
- Use immutable/readonly domain interfaces where practical.
- Prefer additive migrations and backward-compatible route changes.
- Audit consequential actions after successful domain mutation.
- Do not hide preview data behind labels such as Live unless it is genuinely API-backed.

7. Tenancy, Identity and Authorization

7.1 Tenant Resolution

Tenant routes create organisations, while tenant-scoped routes resolve the organisation before authentication. The tenant context is placed into the request and used by repositories and audit events. A token for one tenant must not be accepted against another tenant slug.

7.2 Authentication

- Signup and login require tenant context, email and password.
- The first user in a tenant is bootstrapped as owner; later users default to member.
- JWT sessions contain user ID, tenant ID and roles.
- Role changes do not alter existing JWTs; affected users must sign in again.
- Production must set `SESSION_TOKEN_SECRET`.

7.3 Role and Permission Matrix

Permission	Owner	Staff	Member	Purpose
appointments:book	Yes	Yes	Yes	Create appointments
appointments:manage	Yes	Yes	No	Operational appointment actions
appointments:view	Yes	Yes	No	Organisation appointment visibility
tenant:manage	Yes	No	No	Users, roles and audit access
leave:request	Yes	Yes	Yes	Submit/view own leave and balances
leave:approve	Yes	Yes	No	Review all leave and decide
lifecycle:view	Yes	Yes	Yes	View own or permitted plans
lifecycle:manage	Yes	Yes	No	Create and progress lifecycle plans
cases:create	Yes	Yes	Yes	Open a complaint/case
cases:manage	Yes	Yes	No	List, assign, comment and resolve cases
analytics:view	Yes	Yes	No	View executive summary

Security warning: Role claims are embedded in JWTs. Permission changes require session renewal. Do not add client-side-only authorization or trust roles posted by the browser.

8. Audit, Integrity and Accountability

The audit subsystem records immutable, tenant-scoped facts. Each event hashes its own contents together with the previous event hash, producing a chain that detects mutation or deletion. PostgreSQL appends are protected with a tenant-level advisory lock so concurrent writers cannot fork the chain.

8.1 Representative Audit Actions

Domain	Actions emitted
Tenant/identity	tenant.created, user.signed_up, user.logged_in, user.roles_updated
Appointments	appointment.booked, appointment.checked_in, appointment.completed and other lifecycle actions
Leave	leave.requested, leave.approved, leave.rejected, leave.cancelled
Lifecycle	onboarding.plan_created, offboarding.plan_created, step_completed, plan_cancelled
Cases	case.created, case.assigned, case.status_changed, case.comment_added

8.2 Audit Handling Rules

- Never update or delete an audit event as a correction mechanism.
- Record a new event when correcting an earlier action.
- Do not place passwords, session tokens, full sensitive documents or unnecessary personal data in metadata.
- Include actor, target type, target ID and minimal decision context.
- Run integrity verification during compliance review and incident investigation.
- Back up audit data with the same or stronger retention policy as primary operational records.

9. Runtime Topology and End-to-End Flows

9.1 Organisation Creation and First Login

34. Browser submits organisation details to POST /tenants.
35. Browser signs up the first account under the new tenant slug.
36. Auth service hashes the password and creates the user as owner.
37. JWT is returned and persisted by the web session helper.
38. Tenant and signup audit events are appended.
39. Onboarding UI is shown before the workspace shell.

9.2 Authenticated Request

40. Browser sends tenant slug/context and bearer token.
41. Tenant plugin resolves the tenant.
42. Auth guard verifies token signature and tenant match.
43. Permission pre-handler evaluates server-side roles.
44. Route validates the body/query and calls a domain service.
45. Repository reads/writes only within the tenant ID.
46. Route appends an audit event after successful mutation.
47. Response is returned to the console.

9.3 Leave Request Flow

48. Member submits leave type, start/end dates and reason.
49. Service validates dates and calculates weekdays.
50. Repository checks overlapping pending/approved leave.
51. Service evaluates configured allocation for balance-limited leave.
52. Pending request is stored and audited.
53. Staff/owner lists scope=all and approves or rejects.
54. Decision metadata and balance effects are visible to the requester.

9.4 Customer Case and Executive Flow

55. Any permitted user opens a case with customer email, category and priority.
56. Service generates a tenant-unique reference and priority-based SLA deadline.
57. Staff/owner assigns ownership, comments and changes status.
58. Resolution requires valid state transitions and evidence where enforced.
59. SLA state is derived from deadline and resolution timing.
60. ExecutiveSummaryService aggregates case and appointment records into operational metrics.

10. Domain Module Handover

10.1 Tenancy

- Tenant entity validates organisation slug.
- TenantContext uses AsyncLocalStorage for request-scoped access.
- Postgres enforces slug uniqueness.
- Production gaps: branch/location hierarchy, department/team model, delegated branch administration and database row-level security.

10.2 Identity and Role Administration

- Script password hashing and timing-safe verification.
- JWT signing/verification through jose.
- Tenant-scoped user uniqueness.
- GET /users and PATCH /users/:id/roles are owner-gated.
- Last-owner demotion returns 409.
- Invite, MFA, OIDC/SSO, password reset and session revocation remain outstanding.

10.3 Appointment / Branch Flow

- Appointment booking and lifecycle transitions are implemented.
- Repository supports in-memory and PostgreSQL storage.
- ServiceName remains free text on the mature foundation described in older docs.
- Branch/location, service catalogue, duration, capacity, waitlists, calendar integration and real queues are not active on main.

10.4 Workforce Lifecycle

Concept	Implemented behaviour
Leave types	annual, sick, parental, unpaid, other
Leave states	pending, approved, rejected, cancelled
Rules	Date validation, weekday calculation, overlap prevention, balance enforcement
Balances	Allocated, approved, pending and remaining values by leave type

Concept	Implemented behaviour
Lifecycle kinds	onboarding and offboarding
Lifecycle states	active, completed, cancelled
Steps	Title, owner role, pending/completed, completion actor/time
Storage	leave_requests table; lifecycle_plans with JSONB steps
Authorization	Member self-service; staff/owner approval and lifecycle management
Audit	Request, decision, cancellation, plan and step actions

10.5 Customer Intelligence

Concept	Implemented behaviour
Priorities	low, normal, high, urgent
Statuses	open, in_progress, waiting_customer, resolved, closed
SLA states	on_track, due_soon, breached, met, missed
Case record	Reference, customer email, subject, description, category, priority, owner, deadline and resolution
Comments	Public or internal visibility
Executive metrics	Case totals, active/breached/resolved, SLA compliance, average resolution hours
Appointment metrics	Total, completed, checked-in, no-show, cancelled and completion percentage
Trend	Seven-day cases-created, cases-resolved and appointment counts
Storage	customer_cases and case_comments
Limitation	No external customer portal, omnichannel ingestion, workflow engine or full event-driven BI

11. Complete API Reference

Base URL: Local API defaults to <http://localhost:3000>. The web application normally calls /api through the Vite proxy. Protected requests require tenant context plus a bearer token.

Method	Path	Permission	Input	Result
GET	/health	Public	-	Returns {status:"ok"}
POST	/tenants	Public	tenant payload	Creates organisation
POST	/auth/signup	Tenant	email,password	Creates account; first account owner
POST	/auth/login	Tenant	email,password	Returns JWT/session result
GET	/users	tenant:manage	-	Lists tenant users
PATCH	/users/:id/roles	tenant:manage	roles[]	Updates roles; protects last owner
GET	/audit-events	tenant:manage	-	Returns tenant audit chain
POST	/appointments	appointments:book	appointment fields	Books appointment
GET	/appointments	appointments:view/ manage	query	Lists appointments
POST	/appointments/:id/check-in	appointments:manage	-	Checks in booked appointment
POST	/appointments/:id/complete	appointments:manage	-	Completes checked-in appointment
POST	/leave-requests	leave:request	type,startDate,endDate,reason	Creates pending leave
GET	/leave-requests	leave:request	scope=all optional	Own requests or all with approval permission
GET	/leave-balances	leave:request	-	Returns own balances
POST	/leave-requests/:id/approve	leave:approve	note optional	Approves pending leave
POST	/leave-requests/:id/reject	leave:approve	note optional	Rejects pending leave
POST	/leave-requests/:id/cancel	leave:request	-	Requester cancels allowed request
POST	/lifecycle-plans	lifecycle:manage	subjectUserId,kind,title,dueAt,steps	Creates onboarding/offboarding plan
GET	/lifecycle-plans	lifecycle:view	-	Own or all depending on manage permission
POST	/lifecycle-plans/:planId/steps/:stepId/complete	lifecycle:manage	-	Completes a step and may complete plan
POST	/lifecycle-plans/:id/cancel	lifecycle:manage	-	Cancels active plan
POST	/cases	cases:create	customerEmail,subject,description,category,priority	Creates case with reference/SLA

Method	Path	Permission	Input	Result
GET	/cases	cases:manage	status,priority,customerEmail	Filters cases
GET	/cases/:id	cases:manage	-	Returns case
GET	/cases/:id/comments	cases:manage	-	Returns comments
POST	/cases/:id/assign	cases:manage	ownerUserId null	Assigns/unassigns owner
POST	/cases/:id/status	cases:manage	status,resolution optional	Changes status
POST	/cases/:id/comments	cases:manage	body,visibility	Adds comment
GET	/executive/summary	analytics:view	-	Returns live aggregate summary

11.1 HTTP Error Conventions

Status	Typical meaning	Examples
400	Invalid input or illegal domain transition	Bad leave dates, overlap, insufficient balance, invalid case status
401	Invalid credentials/token or tenant mismatch	Wrong login; tenant A token used for tenant B
403	Authenticated but missing permission	Member requesting all leave or listing cases
404	Tenant-scoped resource not found	Unknown leave, lifecycle plan, case or step
409	Conflict protecting invariant	Attempt to demote last owner
500	Unexpected unhandled failure	Repository/driver defects or programming errors

12. Database, Persistence and Migrations

12.1 Core Tables

Table	Purpose	Tenant isolation
tenants	Organisation identity and slug	Primary tenant record
users	Tenant users, credentials and roles	tenant_id + tenant-scoped email uniqueness
appointments	Customer appointment lifecycle	tenant_id filtered repository operations
audit_events	Immutable hash-chain events	Per-tenant chain and advisory lock

Table	Purpose	Tenant isolation
leave_requests	Leave requests and decisions	tenant_id plus requester/status indexes
lifecycle_plans	Onboarding/offboarding plans and JSONB steps	tenant_id plus subject/status indexes
customer_cases	Complaint/case records and SLA fields	tenant/reference unique plus status/SLA/customer indexes
case_comments	Public/internal comments	tenant_id + case_id + created_at index

12.2 Migration Sequence

Migration	Content	Rule
0001_initial.sql	Tenants, users, appointments and audit events	Foundation; do not rewrite after deployment
0002_workforce_lifecycle.sql	leave_requests and lifecycle_plans	Additive Workforce Lifecycle schema
0003_customer_intelligence.sql	customer_cases and case_comments	Additive cases/SLA schema
0004+	Future domains	Use ordered, immutable and reviewed migration files

12.3 Persistence Operating Rules

- Production must use PostgreSQL and DATABASE_URL.
- In-memory repositories are for development/tests and lose state at process restart.
- Every query for tenant-owned data must include tenantId in the WHERE clause.
- Do not change old migrations to repair deployed environments; create a new corrective migration.
- Test migrations against clean PGLite and a real PostgreSQL service before production.
- Add backup, restore and point-in-time recovery procedures before onboarding paying customers.
- Consider row-level security as defence in depth, not a replacement for application scoping.

13. Web Console Handover

The company console is a React single-page application with local stage management for authentication, onboarding and the application shell. It stores a Klerion session, checks API health, supports a demo session, and maps navigation route keys to view components.

13.1 Current Navigation

Screen	Backing	Handover note
Overview	Preview/partial	Do not rely on displayed branch metrics as live.
Appointments	Live API with historic preview fallback patterns	Complete branch/service scheduling dependencies.

Screen	Backing	Handover note
Live queue	Preview on main	Badge says Live but backend queue is not active.
Users & roles	Live	Owner-gated role administration.
Leave & availability	Live	Self-service and manager actions.
On/Offboarding	Live	Lifecycle plans and steps.
Cases & complaints	Live	Staff case operations.
Executive centre	Live limited	Real cases/appointments aggregate.
Recruitment	Preview	No active API or persistence.
Audit trail	Live	Integrity-aware display.
Reports	Foundation	Not a general reporting engine.

13.2 Session Modes

- **Live mode:** Created by signup/login; API requests use the actual tenant and token.
- **Demo mode:** Creates a local owner-like session with representative records; must never be confused with persisted production data.
- **API indicator:** Shell displays checking/connected/offline or preview state.
- **Sign-out:** Clears stored session and returns to authentication.

13.3 Frontend Technical Debt

- No automated component or end-to-end tests on current main.
- The lint command is TypeScript noEmit rather than ESLint.
- Shared App.tsx and Shell.tsx are conflict hotspots.
- Preview/live labelling is inconsistent with nav badges.
- No generated API client or runtime response validation.
- Accessibility, keyboard navigation and responsive regression need formal testing.
- Package versions should be reviewed for stability and support before production.

14. Security and Privacy Posture

14.1 Controls Already Present

- Tenant-scoped identity and repository access.
- Signed JWT sessions with production secret requirement.
- Script password hashing.
- Server-side RBAC permission guards.
- Last-owner protection.
- Tamper-evident audit chains.
- Tenant-level advisory lock for Postgres audit append.
- Cross-tenant API tests and PGLite integration tests.

14.2 Production Security Gaps

Risk area	Current gap	Required action before high-trust deployment
Session security	No refresh/revocation model documented	Short-lived access tokens, revocation/rotation and secure cookie strategy
MFA/SSO	Not implemented	Add MFA and optional OIDC/SAML for enterprise customers
Network/API hardening	Rate limit/CORS policy/request logging not complete	Add gateway controls, secure headers, rate limits and abuse monitoring
Database defence	No row-level security	Introduce RLS policies and tenant isolation tests
Secrets	Only env-secret requirements documented	Use managed secret store and rotation runbook
Privacy	No retention/deletion/DSR implementation	Create data inventory, retention schedules and deletion workflows
Files	No object storage or malware scanning	Do not accept customer documents until secure file pipeline exists
Observability	Logger disabled; no central error tracking	Enable structured logs, trace IDs and security alerts
Dependency risk	Audit warnings previously noted in dev toolchain	Automate dependency review and patch cadence
Customer-facing cases	Public/internal comment model exists but no external portal	Define data exposure and comment visibility guarantees

15. Local Development Setup

15.1 Prerequisites

- Node.js 24, matching CL.
- npm for root/backend workspaces.
- Corepack-enabled pnpm for apps/web.
- Optional PostgreSQL for persistent local work.
- Git access to the private Lugbinlaye/Klerion repository.

15.2 Clone and Install

```
git clone https://github.com/Lugbinlaye/Klerion.git
cd Klerion
git checkout main
git pull --ff-only
npm ci
cd apps/web
corepack enable
pnpm install
cd ../../
```

15.3 Run in In-Memory Mode

```
# Terminal 1 - API
npm run dev

# Terminal 2 - web console
npm run dev:web

# Open
http://localhost:5173
```

Data persistence: Without DATABASE_URL, restarting the API deletes all locally created tenants, users, appointments, leave plans, cases and audit events.

15.4 Run with PostgreSQL

```
# Example only - use a secure local credential
export DATABASE_URL='postgres://klerion:password@localhost:5432/klerion'
export SESSION_TOKEN_SECRET='replace-with-a-long-random-secret'
npm run dev
```

15.5 First-Use Flow

61. Open the console.
62. Choose Create organisation.
63. Enter a lowercase slug using alphanumeric characters and single hyphens.
64. Create the first user; it becomes owner.
65. Complete or skip the UI onboarding flow.
66. Verify Users & roles, Audit trail, Leave, Lifecycle, Cases and Executive centre.

16. Configuration and Environment Management

Variable	Component	Default	Production rule
DATABASE_URL	API	Unset -> in-memory	Required; managed Postgres connection string
SESSION_TOKEN_SECRET	API	Insecure development fallback	Required; strong managed secret, rotate under runbook
PORT	API	3000	Coordinate with proxy/ingress
VITE_API_BASE_URL	Web	/api	Use relative path behind same-origin gateway where possible
KLERION_API_ORIGIN	Vite build/dev	http://localhost:3000	Set proxy target for environment

16.1 Environment Separation

- Use separate databases, secrets and tenant data for development, staging and production.
- Never point local or preview builds at production by default.
- Do not commit .env files or credentials.
- Use migration approval and backup checkpoints for staging/production.

- Document which frontend build points at which API origin.
- Use environment-specific audit retention and logging destinations.

17. Testing, CI and Quality Gates

17.1 Backend CI

- Runs on pushes and pull requests targeting main.
- Node.js 24 with npm cache.
- npm ci, npm run typecheck, npm run lint, npm test and npm run build.
- Tests include health, vertical slice, RBAC, audit, user roles, workforce lifecycle, customer intelligence and Postgres integration.

17.2 Web CI

- Path-filtered to apps/web and its workflow.
- Uses pnpm 10 and Node.js 24.
- Installs with --no-frozen-lockfile.
- Runs typecheck, static check and production build.
- No behavioural/unit/end-to-end tests are currently required by this workflow.

17.3 Test Strategy

Test type	Mechanism	Strength	Gap
Domain/unit	node:test on services/entities	Fast business-rule coverage	Coverage thresholds not enforced
API integration	Fastify app.inject()	No socket, full route composition	No contract snapshot/OpenAPI validation
Database integration	PGlite/Postgres semantics	Catches real SQL/Drizzle behaviour	Not identical to managed Postgres/network conditions
Web static	TypeScript and Vite build	Catches compile/bundle errors	No interaction, accessibility or visual regression tests
Manual browser	Create org and navigate flows	Confirms integrated UX	Not repeatable or automatically gated

17.4 Definition of Done for New Modules

- Domain model and invariants.
- Tenant-safe repository interface and in-memory/Postgres adapters.
- Ordered migration and indexes.
- Authenticated API with permission checks and input validation.
- Audit events for consequential mutations.
- Live UI without hidden preview fallbacks.
- Unit, API, persistence and cross-tenant tests.
- Documentation and operational notes.
- Both applicable CI workflows green.

18. Git Branch, Pull Request and Merge History

The repository underwent a careful reconciliation on 6 August 2026. Documentation, Developer 4 and Developer 6 work were integrated as active code. Other branches were unsafe to merge directly because of outdated bases, destructive deletions, shared registry conflicts, duplicate migration numbers or unrelated Git history.

18.1 Active Merges

PR / commit	Outcome
PR #3	Six-developer execution plan merged.
PR #4	Workforce Lifecycle merged into main.
PR #7	Customer Resolution and Intelligence reconciled on top of Workforce Lifecycle and merged.
PR #17 / ad86feaa...	Preserved all remaining branch histories with 97 commits and 0 file changes.
40c995c...	Removed the temporary reconciliation workflow; latest reviewed main head.

18.2 History-Only Branches

Branch	Why not activated	How to reuse safely
Developer-1	Large branch/appointment implementation failed CI and changed package/migration strategy	Extract owned files into a new branch from current main; reconcile schema and tests.
Developer-3	Large workforce core implementation based on older main and shared-file changes	Port employee/attendance domain selectively; integrate with Workforce Lifecycle EmployeeRef strategy.
Developer-5	No common history with main	Treat as external source snapshot; reconstruct approved modules into current architecture.
Queue-management	Deleted tenancy, identity, audit, appointments and existing web shell	Never merge directly; salvage queue/notification domain pieces only.
Older Developer 4/6 branches	Superseded by merged conflict-resolved PRs	Use main implementations, not original branch heads.

Branch rule: Every original branch reports zero commits ahead of main because its history is now represented. This does not mean all branch file contents are active. Check the current tree, not ancestry alone, when assessing features.

19. Deployment and Release Runbook

19.1 Recommended Production Topology

- Managed PostgreSQL with backups and point-in-time recovery.
- Containerised or managed Node API service with health checks.
- Static web build served through CDN or same-origin application gateway.
- TLS termination, secure headers, rate limiting and request size limits.
- Managed secrets for DATABASE_URL and SESSION_TOKEN_SECRET.

- Central structured logs, metrics, traces and error reporting.

19.2 Pre-Release Checklist

67. Freeze release scope and identify live versus preview features.
68. Create release branch or tagged commit from clean main.
69. Run backend and web CI.
70. Run database migration in staging from a production-like backup.
71. Execute cross-tenant, authorization and critical user-journey smoke tests.
72. Verify no demo credentials, local URLs or preview records are bundled.
73. Back up production database and confirm restore procedure.
74. Deploy API, verify /health, then deploy web build.
75. Run post-deployment smoke tests and audit-event verification.
76. Tag release and update this handover/manual revision.

19.3 Rollback

- Application rollback: redeploy the previous image/build.
- Database rollback: prefer forward corrective migration; destructive down migrations require explicit tested approval.
- Security rollback: revoke/rotate secrets and invalidate affected sessions.
- Feature rollback: use server-side feature flags for new domains once introduced.
- Data incident: preserve audit/log evidence before corrective changes.

20. Operations, Monitoring and Incident Response

20.1 Minimum Monitoring Before Pilot

Signal	Minimum measure
Availability	API health success rate and web availability
Latency	p50/p95/p99 by route and database query class
Errors	HTTP 4xx/5xx rate, domain-error counts and unhandled exceptions
Authentication	Login failures, tenant mismatch, token verification failures
Database	Connections, locks, slow queries, storage and migration status
Audit	Append failures, chain verification failures and unusual privileged actions
Business	Appointments, pending leave, breached cases and resolution time

20.2 Incident Severity Model

Severity	Example	Initial action
SEV-1	Cross-tenant data exposure, credential	Stop affected service, preserve

Severity	Example	Initial action
	compromise, widespread outage	evidence, notify security/owners immediately
SEV-2	Major domain unavailable, migration failure, audit append failures	Rollback or isolate feature; start incident channel and status updates
SEV-3	Limited user-impacting defect with workaround	Triage, patch under normal emergency release process
SEV-4	Cosmetic/preview/content issue	Backlog and fix in scheduled release

20.3 Incident Checklist

77. Confirm tenant(s), users, environment and first observed time.
78. Capture logs, request IDs, deployment SHA, database state and audit-chain status.
79. Contain access or stop harmful writes.
80. Assess cross-tenant or privacy impact.
81. Communicate scope and workaround.
82. Repair through reviewed code/migration, not direct untracked database edits.
83. Verify recovery and audit integrity.
84. Complete post-incident review with preventive actions.

21. Known Limitations, Risks and Technical Debt

Risk	Priority	Impact	Recommended treatment
Documentation drift	High	PROGRESS.md understates active modules and can mislead planning	Update immediately and automate release notes
Preview/live mismatch	High	Queue nav badge and polished screens can overstate capability	Remove or clearly gate preview modules for pilots
Tenant isolation application-only	High	Missing tenant filter could expose data	Add RLS, repository tests and security review
No observability	High	Production failures may be hard to detect/diagnose	Structured logs, metrics, traces and alerts
No web behavioural tests	High	UI regressions can ship despite green build	Add React tests, Playwright and accessibility checks
Two package managers	Medium	Install/CI complexity and inconsistent linting	Converge or formally document ownership
Shared registries	Medium	Parallel branches conflict in server/context/App/Shell/permissions	Implement route/feature registries and CODEOWNERS
No branch/service domain	High	Blocks real queue and capacity routing	Build organisation/branch/service foundation first
No background jobs	Medium	Notifications/escalations cannot run reliably	Add durable queue/scheduler before automation
No customer portal	Medium	Case visibility and public	Design authenticated customer

Risk	Priority	Impact	Recommended treatment
		comments are staff-only	access and privacy rules
Migration governance	Medium	Long-lived environments need explicit history/rollback discipline	Add migration table/tooling and staging validation
Session revocation/MFA absent	High for enterprise	Stolen JWT remains valid until expiry	Add short-lived tokens, refresh/revocation and MFA

22. Six-Developer Ownership Model

Developer	Owned product area	Active state on main
Developer 1	Branches/services and appointments	Appointments foundation active; branch/service work historic only
Developer 2	Queues and notifications	Historic Queue-management work not active
Developer 3	Employee records and attendance	Historic Developer-3 work not active
Developer 4	Leave plus onboarding/offboarding	Active and merged
Developer 5	Forms/workflow/approvals/internal service desk	Historic unrelated work not active
Developer 6	Cases/SLA and executive intelligence	Active and merged

22.1 Conflict-Prevention Rules

- Each developer owns exclusive feature folders and schema/migration files.
- Cross-domain references use published IDs/contracts, not direct table mutation.
- Developer 1 or designated integrator owns root configuration, shared registries and CI.
- Every branch starts from latest main and stays current through small, frequent integrations.
- Large generated documentation folders and build artifacts must not be committed.
- No branch may replace package-manager strategy or delete core modules without an architecture decision record.
- Use stacked PRs only when dependency order is explicit and CI remains understandable.

23. Commercial Roadmap and Recommended Delivery Order

23.1 Strongest 12 Commercial Modules

Priority	Module	Current state
1	Virtual Queue and Remote Check-In	Not active on main
2	Smart Appointment Scheduling	Foundation active
3	Request and Approval Workflow	Not active

Priority	Module	Current state
	Engine	
4	Time and Attendance	Historic only
5	Internal Service Desk	Not active
6	Branch and Service Discovery with Capacity Routing	Not active
7	Employee Master Records and Directory	Historic only
8	Case, Complaint and SLA Management	Active
9	Omnichannel Notifications	Not active
10	Leave and Availability	Active
11	Digital Onboarding and Offboarding	Active
12	Executive Command Centre and BI	Active limited

23.2 Recommended 30/60/90-Day Plan

Period	Objectives	Exit criteria
Days 0-30	Documentation refresh; branch/service domain; schema and route registry; web test foundation; observability baseline	Current docs accurate; branches/services persisted; green CI; structured logs and request IDs
Days 31-60	Complete appointment availability; selectively integrate employee core; queue domain and SSE snapshot; notification provider contract	Real branch appointment journey; employee records; real queue API; provider-independent notifications
Days 61-90	Pilot hardening; customer remote check-in; SLA escalations; executive cross-domain metrics; deployment/security controls	One industry pilot journey has no preview data; production checklist and incident runbook proven

23.3 Recommended First Pilot

The strongest first pilot remains Klerion Flow for a service-centre environment, but it should not launch until branch/service discovery, appointment capacity, queue tickets, remote check-in and notifications are real. An alternative near-term pilot is Klerion Resolve & Insight because cases and executive summaries are already active; this requires customer intake, SLA policy configuration, stronger observability and a controlled implementation partner.

24. Troubleshooting Guide

Symptom	Likely cause / action
Web shows Offline	Confirm API on port 3000; check KLERION_API_ORIGIN and Vite proxy; call /health directly.
Cannot sign in after role change	Sign out and sign back in because roles are embedded in the JWT.
Last owner update returns 409	Promote another user to owner before removing the final owner role.
Data disappears after restart	DATABASE_URL is unset and in-memory repositories are active.
Web port changes/fails	Vite uses strict port 5173; stop the existing process rather than accepting another port.
Tenant A token rejected on tenant B	Expected security behaviour; use matching tenant slug and session.
Leave request rejected	Check date order, weekday count, overlaps and remaining allocated balance.
Case status rejected	Follow allowed state transitions and provide resolution when required.
Migration failure	Stop deployment, capture error, verify backup, test corrective forward migration in staging.
CI typecheck TS6305	Remove committed build artifacts/tsbuildinfo and verify project references/build order.
Queue feature missing on main	Queue branch history is preserved only; do not assume its files were activated.
Developer-5 PR cannot merge	Its history was unrelated; reconstruct approved code from current main.

25. Formal Handover Checklist

Checklist item	Acceptance evidence
Repository access	Incoming lead can clone private repository and administer branches/PRs.
Main branch	Incoming lead confirms current main SHA and branch protection rules.
CI access	Incoming lead can view backend and web workflow logs and rerun failed jobs.
Local run	API and web console run successfully on a clean machine.
Production secrets	Ownership and rotation procedure identified; no secret

Checklist item	Acceptance evidence
	included in this document.
Database	Environment URLs, backup owner and restore procedure documented separately.
Live/preview scope	Product and sales teams acknowledge the feature truth matrix.
Security	Tenant isolation, JWT, permission and audit assumptions reviewed.
Domain ownership	Six-developer responsibilities and shared-file owner accepted.
Branch history	Team understands history-only merge versus active file content.
Roadmap	Next 30-day objectives approved.
Documentation	PROGRESS.md and DEVELOPMENT.md update assigned.
Pilot	Pilot industry, tenant and success metrics selected.
Support	Incident contact and escalation path established.
Acceptance	Section 26 signed by outgoing and incoming owners.

26. Acceptance Sign-Off

Signing this section confirms that the incoming owner has received repository access, reviewed the current production truth, understands the security and operational limitations, and accepts responsibility for future changes from the stated main commit.

Role	Name	Signature	Date
Outgoing project owner			
Incoming engineering lead			
Product owner			
Operations/DevOps owner			
Security reviewer			

Appendix A Command Reference

Backend

```

npm ci
npm run dev
npm run typecheck
npm run lint
npm test
npm run build

# Web
npm run dev:web
npm run build:web
cd apps/web && corepack pnpm typecheck && corepack pnpm lint && corepack pnpm build

# Git baseline
git checkout main
git pull --ff-only
git checkout -b feature/<domain>-<short-name>

```

Appendix B Repository Source Index

Path	Why it matters
DEVELOPMENT.md	Local operation, configuration, testing approach and architectural rules
PROGRESS.md	July foundation status; now partially stale
docs/TEAM_EXECUTION_PLAN_6_DEVELOPERS.md	Commercial scope, team ownership and dependency rules
apps/api/src/server.ts	HTTP route composition
apps/api/src/context.ts	Service/repository wiring
modules/platform/identity/src/permission.ts	Roles and permissions
apps/api/src/routes/workforce-lifecycle.ts	Leave/lifecycle API
apps/api/src/routes/customer-intelligence.ts	Case/executive API
apps/api/src/domains/workforce-lifecycle/**	Workforce domain and storage
apps/api/src/domains/customer-intelligence/**	Case/SLA and executive domain
packages/persistence/migrations/0002_workforce_lifecycle.sql	Workforce tables
packages/persistence/migrations/0003_customer_intelligence.sql	Case tables
apps/web/src/App.tsx	View routing and session stages
apps/web/src/components/Shell.tsx	Navigation and global shell
.github/workflows/ci.yml	Backend quality gate
.github/workflows/web-ci.yml	Web quality gate

Appendix C Audit Event Catalogue

Action	Meaning
tenant.created	Organisation created
user.signed_up	Account created
user.logged_in	Successful login
user.roles_updated	Roles changed with previous-state metadata
appointment.booked	Appointment created
appointment.checked_in	Customer checked in
appointment.completed	Appointment completed
leave.requested	Leave submitted
leave.approved	Leave approved
leave.rejected	Leave rejected
leave.cancelled	Leave cancelled
onboarding.plan_created	Onboarding plan created
offboarding.plan_created	Offboarding plan created
onboarding.step_completed / offboarding.step_completed	Checklist step completed
onboarding.plan_cancelled / offboarding.plan_cancelled	Lifecycle plan cancelled
case.created	Case opened
case.assigned	Case owner changed
case.status_changed	Case lifecycle changed
case.comment_added	Public/internal comment added

Appendix D Glossary

Term	Definition
Tenant	An organisation isolated within the Klerion SaaS environment.
Tenant context	Request-scoped resolved tenant identity used by repositories and authorization.
RBAC	Role-based access control mapping owner/staff/member

Term	Definition
	roles to permissions.
Vertical slice	Domain model, storage, API, authorization, audit, UI and tests delivered together.
PGlite	PostgreSQL compiled to WebAssembly, used for realistic database tests.
SLA	Service-level agreement deadline applied to a customer case.
SLA state	Derived state: on track, due soon, breached, met or missed.
Lifecycle plan	Onboarding or offboarding checklist for a subject user.
History-only merge	Git merge that preserves branch ancestry while keeping the current production tree.
Preview data	Representative UI data not backed by the production domain/API.
Main	Default integration branch and source of the active code tree.

Appendix E Recommended New ADRs

- 85. ADR: Branch, service and organisation hierarchy.
- 86. ADR: Frontend package-manager convergence and test framework.
- 87. ADR: Cross-domain contracts and event envelope.
- 88. ADR: Migration versioning and rollback policy.
- 89. ADR: Authentication refresh, revocation, MFA and SSO.
- 90. ADR: Queue realtime transport and command idempotency.
- 91. ADR: Observability and incident correlation standard.
- 92. ADR: Tenant row-level security strategy.
- 93. ADR: Background job and notification provider architecture.
- 94. ADR: Preview/demo feature governance.

Appendix F Suggested Pilot Success Metrics

Area	Metric	Example target
Customer flow	Average wait time	Reduce by at least 20% from baseline
Appointments	No-show rate	Reduce by 15% after reminders
Cases	SLA compliance	At least 90% within configured SLA
Cases	Average resolution time	Month-on-month reduction
Workforce	Leave decision time	Median under one business day
Lifecycle	Checklist completion	95% by due date

Area	Metric	Example target
Reliability	API availability	At least 99.5% during pilot hours
Security	Cross-tenant incidents	Zero
Adoption	Weekly active staff	At least 80% of designated pilot users
Data quality	Preview records in pilot flow	Zero

Appendix G Pre-Production Security Review Questions

- Can every tenant-owned query prove tenant scoping in code and tests?
- Are JWT lifetimes, rotation and revocation acceptable for the customer risk profile?
- Are secrets stored and rotated outside source control?
- Can audit events be verified and retained without exposing sensitive content?
- Are logs free from passwords, tokens and excessive personal data?
- Are database backups encrypted and restore-tested?
- Are rate limits and abuse controls enabled?
- Are public/internal case comments exposed only to intended audiences?
- Are preview/demo features disabled or unmistakably labelled?
- Has an independent penetration test or security review been scheduled before enterprise production?

Appendix H Final Handover Statement

The Klerion repository is a credible, extensible platform foundation with several live administrative vertical slices. It is not yet the full 12-module commercial suite shown in the product vision. The safest continuation strategy is to preserve the current modular-monolith patterns, close documentation and observability gaps, complete the branch/service foundation, and integrate historic developer work selectively through small, CI-verified pull requests. The project should be sold and piloted according to verified live capability, with preview modules clearly separated from operational commitments.

END OF HANDOVER MANUAL

Repository: Lugbinlaye/Klerion | Reviewed at main 40c995c4ace44714c41c9f03b8c614d987446d54